

WIC API Functionality

General Information [🔗](#)

The API can be used to get data from the WHINT Integration Cockpit in JSON format.

[General Information](#)

[Most Used Entities](#)

[Usage of the API](#)

[Reading Data from the WIC](#)

[Select specific data](#)

[Filtering](#)

[Add/Join other tables](#)

[Write data to the API](#)

[Create a new inventory item](#)

[Update an inventory item](#)

[Additional Information](#)

Most Used Entities [🔗](#)

- inventory
- task
- datasource
- dataFlow
- system
- logEntry

Usage of the API [🔗](#)

The API is build similar to SQL and requests can be build with filters and options like in SQL.

This document will explain the base functionality of the API and possible options.

Reading Data from the WIC [🔗](#)

To read data from the WHINT Integration Cockpit via API, we have to build a JSON query.

As an Example, the SQL command to get all data from the table “inventory”:

```
1 select * from inventory
```

To get the same data via the API, this is transformed to a JSON-Query like this:

```
1 {  
2   "query": {  
3     "entity": "inventory"  
4   }  
5 }
```

Basically, both request will response with the same data from the database only the format is different.

The API response will give you all entries from the database “system” in JSON:

```
1 [
```

```

2      {
3          "id": "IV0S968v_0s",
4          "create_time": "2024-08-05T12:56:18Z",
5          "change_time": "2024-10-10T15:13:43Z",
6          "type": 21,
7          "name": "Issue-Tracking-System >> JIRA Service-Desk 6.4.14",
8          "object_id": "08d2f431-1228-4443-8e15-8df9f4dd28f5",
9          "sender_id": "01u_pLIiqVg",
10         "sender_name": "Issue-Tracking-System",
11         "data_source_id": "w20P2A"
12     },
13     ....
14 ]

```

The response contains all data of each inventory item. The containing data can be different for each object type.

Select specific data [🔗](#)

It is possible to add specific fields for the response.

The SQL the request would look like this:

```

1 select id,type,name from inventory

```

In order to get the same data via the API, we have to add the option “fields” to the JSON query:

```

1 {
2     "query": {
3         "entity": "inventory",
4         "fields": ["id","type","name"]
5     }
6 }

```

The Response for this request contains only the selected fields:

```

1 [
2     {
3         "id": "IV0S968v_0s",
4         "type": 21,
5         "name": "Issue-Tracking-System >> JIRA Service-Desk 6.4.14"
6     }

```

The available fields can be different for each type.

Filtering [🔗](#)

Filtering the results can also be useful in order to get more specific results from the WHINT Integration Cockpit.

In the SQL command, we simply add a “where” clause to the command like this:

```

1 select * from inventory where type eq 21

```

For the API we can add also a “where” option to the JSON query:

```

1 {
2     "query": {
3         "entity": "inventory",
4         "fields": [],
5         "where": [{
6             "option": 0,
7             "conditions": [
8                 {

```

```

9      "field": {
10        "name": "type",
11        "eq": "21"
12      }
13    }
14  ]
15 }
16 }
17 }

```

In the filter we can set the parameter “option” to “0” or “1”.

- “0” = “AND”
- “1” = “OR”

In this case the API response with all items from inventory of the type “21” (EAM).

```

1  [
2    {
3      "id": "IV0S968v_0s",
4      "create_time": "2024-08-05T12:56:18Z",
5      "change_time": "2024-10-10T15:13:43Z",
6      "type": 21,
7      "name": "Issue-Tracking-System >> JIRA Service-Desk 6.4.14",
8      "object_id": "08d2f431-1228-4443-8e15-8df9f4dd28f5",
9      "sender_id": "01u_pLIiqVg",
10     "sender_name": "Issue-Tracking-System",
11     "data_source_id": "w20P2A"
12   },
13   ...
14 ]

```

Add/Join other tables [🔗](#)

Like in SQL, it is possible to join other tables in the request.

SQL would look like this:

```

1 select * from inventory where type eq 21 INNER JOIN system ON inventory.data_source_id = dataSource.id;

```

Transformed into the JSON query, the option “with” is added:

```

1  {
2    "query": {
3      "entity": "inventory",
4      "fields": [],
5      "where": [{
6        "option": 0,
7        "conditions": [
8          {
9            "field": {
10              "name": "type",
11              "eq": "21"
12            }
13          }
14        ]
15      }],
16      "with": [

```

```

17     {
18         "entity": "dataSource",
19         "fields": ["id", "name"]
20     }
21 ]
22 }
23 }

```

In the Join, it is also possible to select specific fields from the additional database.

In this example the fields “id” and “name” are selected.

This will show additional data from the datasource in the response:

```

1  [
2    {
3      "id": "IV0S968v_0s",
4      "create_time": "2024-08-05T12:56:18Z",
5      "change_time": "2024-10-10T15:13:43Z",
6      "type": 21,
7      "name": "Issue-Tracking-System >> JIRA Service-Desk 6.4.14",
8      "object_id": "08d2f431-1228-4443-8e15-8df9f4dd28f5",
9      "sender_id": "01u_pLIQVg",
10     "sender_name": "Issue-Tracking-System",
11     "data_source_id": "w20P2A",
12     "data_source": {
13       "id": "w20P2A",
14       "name": "WHINT LeanIX"
15     }
16   },
17   ...
18 ]

```

The additional data is shown in the response here:

```

1  "data_source": {
2    "id": "w20P2A",
3    "name": "WHINT LeanIX"
4  }

```

The available fields can be different for each database.

Write data to the API [🔗](#)

The API also provides the possibility to add data to the WHINT Integration Cockpit.

Create a new inventory item [🔗](#)

Like the read of the API, the write is also similar to SQL.

In SQL a simple insert would look like this:

```

1  insert into inventory (name,description,type,..)
2  values ("Postman_API","Postman inbound connection", 16,..)

```

To use the same command with the API, it has to be an upsert like this:

```

1  {
2    "upsert": {

```

```

3     "inventory": [
4       {
5         "name": "Postman | API 2",
6         "description": "Postman inbound connection to the WHINT integration Cockpit API",
7         "type": 16
8       }
9     ]
10  }
11 }

```

In the response, it will show the id of the new inventory item.

```

1  [
2    "N6YcJ-SUEgo"
3  ]

```

The result in cockpit will look like this:

Postman_API

Basic Data	
Name	Postman_API
Type	SAP_EVENTMESH
Sender	+
Receiver	+

Description

Postman inbound connection to the WHINT integration Cockpit API

Properties	
☐ Type	Value
no data to display	

Tags	
☐ Type	Value
no data to display	

Objects	
☐ Name	Value
no data to display	

Update an inventory item [↗](#)

To add data to the new created item, an upsert can be used like this in SQL:

```

1 UPSERT INTO inventory (id, name, type)
2 VALUES (N6YcJ-SUEgo, 'name', 'type');

```

As a JSON query it would look like this for the API:

```

1 {
2   "upsert": {
3     "inventory": [
4       {
5         "id": "N6YcJ-SUEgo",
6         "name": "Postman Inbound API",
7         "type": 15
8       }
9     ]
10  }
11 }

```

Response:

```

1  [
2    "N6YcJ-SUEgo"
3  ]

```

With this update, the name and type of the item is updated.

Postman Inbound API		
Basic Data		
Name	Postman Inbound API	
Type	SAP_SOAP	
Sender	▼	
Receiver	▼	

WIC view

Additional Information

 List of all inventory types: [WIC API Inventory Types](#)

List of all Entities: [WIC API Entities](#)

More examples: [WIC API Examples](#)

Postman Collection: [WHINT Integration Cockpit API.postman_collection.json](#)